Assignment 4
Rendering Engine Development

Tanks Game Engine | C++17 and SFML 2.6.2 | 13 November 2026

Source repository: github.com/yussik22/tanks-game-engine

What I implemented

I added an SFML window for the existing 13 x 13 tank game. The game rules remain in GameEngine and GameMap. The renderer reads the same map state as the console application, converts each cell to the familiar console symbol, and draws a corresponding pixel texture. The player and bot sprites rotate to show their current facing direction.

Design at a glance

Input	Game logic	Display
SFML key event -> WindowInput -> KeyboardInput	Command -> GameEngine -> GameMap and CollisionSystem	GameDisplayData -> SpriteRenderer -> SFML window

The graphic application is a separate CMake target, **tanks_render**. The console target, **tanks_cli**, still uses the same engine library. This keeps rendering concerns outside the gameplay classes.

Visual check: Ready state

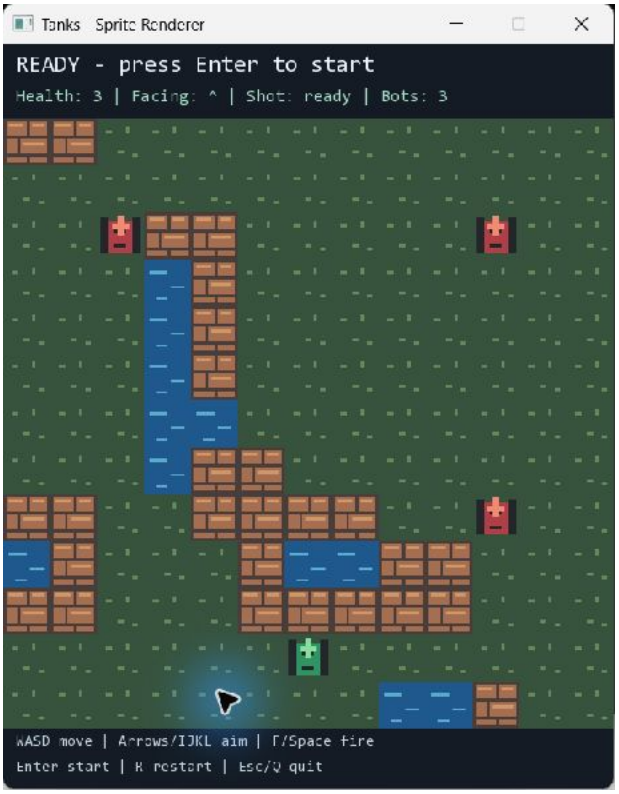


Figure 1. Ready screen: generated map, health, facing and reload status.

Implementation and verification

Rendering pipeline

File	Responsibility
GameDisplayData	Reads objects and produces one console-style symbol per map cell, plus state and HUD text.
PixelTextures	Creates 16 x 16 pixel images for floor, water, wall, tanks and projectile; loads them into SFML textures.
SpriteRenderer	Maps symbols to textures, scales them to cells, rotates tank sprites, and draws the HUD.
WindowInput	Converts SFML key events and held WASD keys to existing game commands.
graphics/main.cpp	Owns the SFML window, event loop, clock, engine and renderer.

For example, an intact wall becomes #, a damaged wall becomes **+**, and the renderer chooses the matching texture. Projectiles and tanks are handled by the same symbol conversion, so the graphic view follows the console game's object priority.

Visual check: running battle

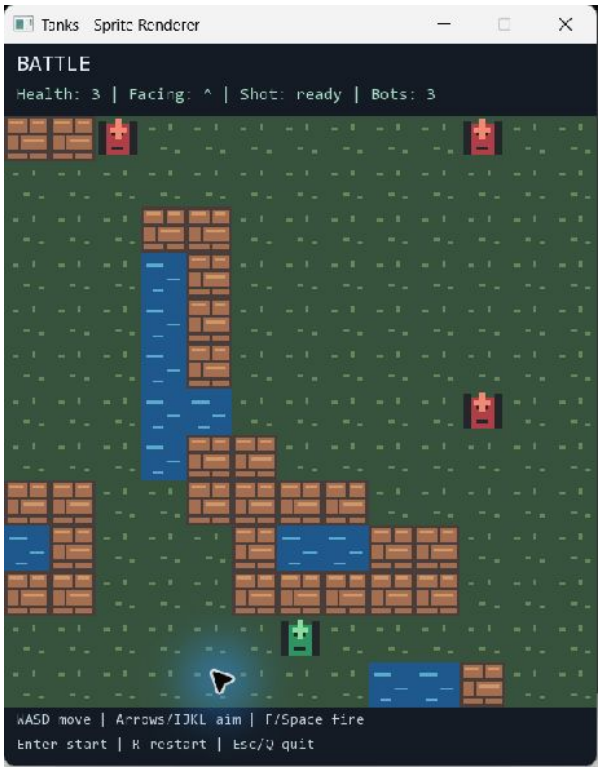


Figure 2. Battle state after Enter: bots have taken actions; map and HUD remain visible.

Build and tests: both the SFML and core-only CMake configurations built successfully. CTest reported 1/1 test target passed in each configuration. The engine tests cover map generation, walls, water, tank and projectile collisions, bot decisions, game states, input timing, and the capped long-frame update.

Control questions

1. How are OOP principles used in the rendering engine?

Abstraction and polymorphism: Renderer declares a virtual render(const GameEngine&) operation. ConsoleRenderer and SpriteRenderer implement it differently. The graphical renderer uses SFML's sf::RenderWindow, sf::Sprite, sf::Texture and sf::Text objects. **Encapsulation:** PixelTextures owns its textures; SpriteRenderer hides drawing helpers and its window reference. GameDisplayData is a small value object passed between reading game state and drawing. Gameplay does not call SFML directly.

2. Is SFML written using OOP? How can that be shown?

Yes. Its public API exposes classes with state and methods, such as sf::RenderWindow, sf::Texture and sf::Sprite. sf::Sprite is a drawable object; the application passes it to window.draw(sprite). SFML also provides inheritance-based interfaces such as sf::Drawable. This can be checked directly in the SFML headers and in our use of those types.

3. Have you used a design pattern?

The two Renderer implementations provide **Strategy-style substitution**: they share one rendering contract, while the console and graphical applications choose different implementations. The current application selects a renderer in its respective main function, so there is no runtime switching. GameMap::generate(seed) is a **simple factory function** that constructs a complete map. It is not the GoF Factory Method pattern. Commands are a struct with an enum, not GoF Command objects.

Representative code

```
class Renderer
{
public:
    virtual ~Renderer() {}
    virtual void render(const GameEngine& engine) const = 0;
};

class SpriteRenderer : public tanks::Renderer
{
    void render(const tanks::GameEngine& engine) const override;
};
```

Setup and compilation

4. How was SFML set up? Dynamic or static libraries?

CMake FetchContent obtains SFML 2.6.2 when TANKS_BUILD_RENDERER=ON. The tanks_render executable links to sfml-graphics, which brings in sfml-window and sfml-system. On Windows, CMake copies their DLLs beside the executable. This build uses **dynamic SFML libraries** (DLLs); the project's own tanks_engine library is static. Dynamic linking keeps library code in separate runtime files, while static linking copies the required library code into the executable at link time. A dynamic build needs compatible DLLs at run time.

```
cmake -S . -B build-render -DTANKS_BUILD_RENDERER=ON
cmake --build build-render --config Release --target tanks_render
ctest --test-dir build-render -C Release --output-on-failure
```

5. Why are header files needed at compilation, while libraries are needed later?

Each .cpp file is compiled separately. Headers give the compiler declarations: class layouts, function signatures and types needed to check calls and generate object files. A library supplies compiled definitions for the linker to resolve external symbols. SFML headers alone therefore let the code compile but cannot provide the implementations of sf::RenderWindow or sf::Texture. Inline and template definitions are the main exception: their definitions must be visible in the translation unit, usually through a header.

How to run and reproduce

On Windows, launch build-render/Release/tanks_render.exe after building. Press Enter to start, hold WASD to move, use arrow keys or IJKL to aim, F or Space to fire, and R after a win or loss to restart. Esc closes the window. The screenshot files in report/images are captured from the built graphical application.

Current scope

The tank has one facing direction: movement also turns the tank and changes its firing direction. The graphic renderer turns the whole tank sprite. Separate hull and turret angles are not part of this version. The renderer loads a system monospace font, so a machine without one of the configured font paths would need an added font fallback.

SFML documentation: sfml-dev.org/documentation/2.6.2